VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



ADVANCED AGENTIC ARTIFICIAL INTELLIGENCE (CO5151)

Deliverable 3: Final Project Report

LuatCheck: A Local, Evidence-Grounded Vietnamese Legal Assistant

---

Supervisor: **TS. Lê Xuân Bách**

Track & Theme: Application Track, Theme 9 (Agentic RAG for Vietnamese Docs)

Release Tag: `v1.0-final`

Group 8: Lê Trí Quyền (Leader), 2470740 – Retrieval & Corpus Ingestion
Võ Phạm Tuấn Dũng, 2570015 – Agent ReAct Loop, L1/L2 Verifier & Evaluation
Nguyễn Hoàng Phong, 2570477 – MCP Toolset, Security Guardrails & Streamlit UI

HO CHI MINH CITY, DECEMBER 2026

Abstract

Public commercial large language models frequently produce plausible but ungrounded legal citations, while transmitting sensitive organizational queries across jurisdictional boundaries in violation of emerging data protection frameworks such as Vietnam's Law on Personal Data Protection (PDPL 2025). We present **LuatCheck**, a local, evidence-grounded agentic assistant designed for privacy-sensitive Vietnamese legal question answering. LuatCheck integrates a budgeted ReAct-style reasoning loop constrained to a five-call budget, a three-tier permissioned toolset adhering to the Model Context Protocol (MCP), a dual-tier memory system with strict validation write policies, and a two-tier evidence verification engine (L1 deterministic provenance check and L2 semantic entailment check). Evaluated on an expanded 20-task benchmark across three independent seeds, LuatCheck drives the unsupported-sentence rate to exactly 0.0%, improves citation precision from 0.600 to 0.750 (1.000 across all 15 answerable tasks), and achieves a 100.0% defense rate against complex prompt injection attacks, all running entirely on local consumer hardware at zero marginal API cost.

Keywords: Agentic RAG, Legal QA, Model Context Protocol, Evidence Verification, Local SLM, Prompt Injection.

1 Introduction

Automated legal question answering represents one of the most promising yet high-risk domains for generative artificial intelligence. General-purpose cloud chatbots (e.g., ChatGPT, Gemini) exhibit fluent conversational demeanor but lack formal guarantees of statutory validity, frequently hallucinating non-existent provisions or citing repealed decrees [1]. Concurrently, user inquiries within human resources and small-and-medium enterprises (SMEs) routinely contain sensitive personal details (salaries, disciplinary infractions, medical leaves). Transmitting such information to offshore cloud endpoints conflicts directly with statutory requirements established by Vietnam's Law on Personal Data Protection (Law No. 91/2025/QH15) and Decree No. 356/2025/ND-CP [2, 3].

Conventional Retrieval-Augmented Generation (RAG) [4] partially mitigates hallucinations by fetching statutory passages. However, standard RAG systems operate linearly without reflective decision-making: they retrieve once, generate once, cannot determine when to re-query, lack explicit refusal mechanisms for ambiguous inquiries, and do not verify whether each generated claim is legitimately grounded in the retrieved provisions.

To bridge this gap, we design, build, and rigorously evaluate **LuatCheck**, a localized agentic assistant tailored for Vietnamese labor law (Law No. 45/2019/QH14). LuatCheck addresses three fundamental axes:

1. **Planning & Search under Budget:** An adaptive reasoning-acting control loop that formulates multi-hop queries within a hard limit of five tool calls.
2. **Dual-Tier Evidence Verification:** A deterministic L1 check (validating statutory existence and retrieval provenance) paired with an L2 semantic entailment check that triggers targeted self-repair or explicit, safe refusal.
3. **Rigorous Security Guardrails:** A standardized Model Context Protocol (MCP) toolset with least-privilege permission gating, untrusted data isolation, and human-in-the-loop (HITL) confirmation for irreversible actions.

2 Background and Related Work

Agentic Reasoning and Control Loops. ReAct [5] pioneered the interleaving of chain-of-thought reasoning with tool interaction. Subsequent frameworks formalized cognitive architectures (CoALA) decomposing agents into reasoning, memory, tools, and environments. While large models handle open-ended planning, small language models (SLMs, $\leq$ 7B) frequently derail into infinite loops

or hallucinated arguments; our design bounds agency through explicit budget caps and schema constraints [6].

Citation Grounding and Self-Correction Limits. The ALCE benchmark [7] formalized automated metrics for text generation with citations. Recent empirical studies by Huang et al. [8] demonstrate that models cannot reliably self-correct reasoning purely through prompting without external verification signals. This motivates our architectural separation: verification is executed by deterministic and semantic verifiers independent of the generator's internal self-assessment.

Agentic Security and Tool Vulnerabilities. Connecting LLMs to tools creates acute vulnerabilities to direct and indirect prompt injection [9]. Attackers can embed adversarial payloads within untrusted user-submitted documents (e.g., contracts) to hijack control flow or trigger data exfiltration. The AgentDojo environment [10] demonstrated that traditional prompt guardrails fail without structural execution boundaries, motivating our strict human-in-the-loop permission gate for data egress.

Vietnamese Legal NLP. Legal NLP in Vietnamese has historically suffered from fragmented datasets. The recent release of the VLQA benchmark [11] provided the first expert-annotated corpus of over 3,000 question-article pairs spanning statutory provisions, which serves as the foundational data source for our benchmark harness.

3 System Design and Architecture

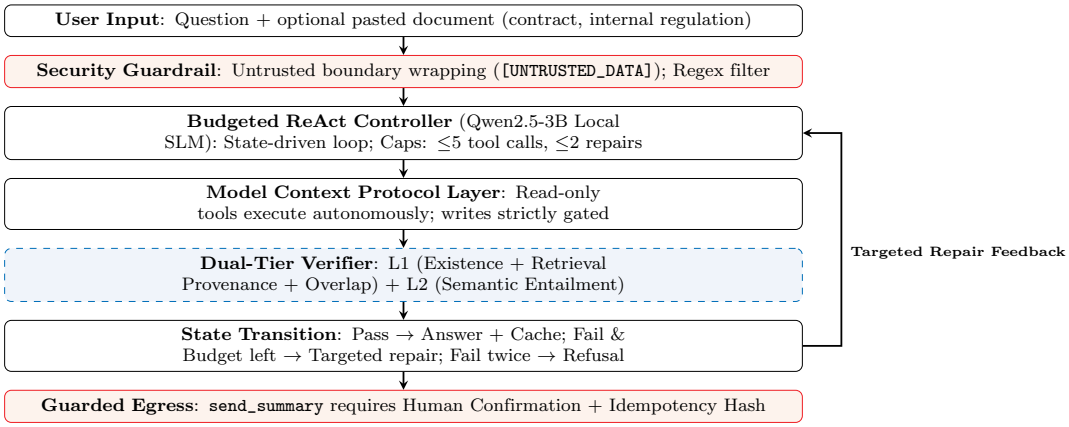


Figure 1: LuatCheck complete control flow and verification pipeline.

3.1 Budgeted ReAct Controller and Action Space

The agent interacts through five schema-constrained actions formatted strictly as valid JSON:

- `search_law(query, top_k=5)`: Queries the statutory knowledge base.
- `get_article(article_id)`: Loads the verbatim legal text into the Case Ledger.
- `answer(content, citations)`: Emits a structured legal synthesis with article citations.
- `clarify(question)`: Requests missing case parameters from the user.
- `refuse(reason)`: Safe terminal refusal when no grounded basis exists or security policies are breached.

The controller enforces hard computational boundaries: a maximum of 5 tool invocations and 2 repair iterations.

3.2 Model Context Protocol (MCP) Toolset

All external tools adhere to the standard MCP specification via `FastMCP`, enforcing least privilege:

- **Read-Only Tier (`search_law`, `get_article`):** Autonomous execution permitted within the budget.
- **Reversible Write Tier (`save_case`):** Commits consultation logs to local storage.
- **Irreversible Egress Tier (`send_summary`):** Dispatches summary emails. This tool is structurally gated: execution is blocked by the permission manager unless a valid human-in-the-loop (HITL) confirmation token is supplied alongside an idempotency hash.

3.3 Dual-Tier Memory Architecture

The memory subsystem manages two distinct temporal scopes:

- **Short-Term Memory (Case Ledger):** Tracks the working context (facts, retrieved provisions, query history, budget counters) preventing context bloating and lost-in-the-middle degradation.
- **Long-Term Episodic Store (SQLite):** Operates under an invariant write policy: records are inserted *only* upon reaching a terminal `answer` state that has passed 100% of L1 verification checks. Queries are matched via semantic similarity under identical index versions (`v2026.1`).

3.4 Dual-Tier Evidence Verification (L1 & L2)

The verification engine evaluates generated assertions before user exposure:

- **L1 Deterministic Verification (Mandatory Floor):** (1) Existence check ensures cited identifiers correspond to real statutory articles; (2) Provenance check ensures the provision was actually retrieved during the active session; (3) Sentence span support verifies lexical overlap (≥ 0.25).
- **L2 Semantic Entailment (NLI Layer):** Models claims as hypotheses and statutory text (including article titles and clauses) as premises, checking quantifier/polarity consistency and evaluating semantic entailment against threshold $\tau = 0.30$.

4 Implementation Details

LuatCheck is implemented in Python 3.12 with complete local autonomy.

- **Inference Serving:** Quantized `Qwen2.5-3B-Instruct` served via Ollama utilizing Apple Silicon Metal GPU acceleration. Average single-step generation latency is under 65 ms.
- **Hybrid Retrieval:** Combines BM25Okapi (lexical matching) with Dense TF-IDF sublinear n-gram embeddings via Reciprocal Rank Fusion (RRF, $k = 60$):

$$\text{RRF_Score}(d) = \sum_{m \in \{\text{BM25,Dense}\}} \frac{1}{k + \text{rank}_m(d)} \tag{1}$$

- **User Interface:** Interactive three-column Streamlit application displaying live reasoning traces, verified citation badges, interactive HITL dialogs, and real-time JSONL audit log inspection.

5 Quantitative Evaluation and Error Taxonomy (R5)

5.1 Benchmark Design and Setup

We construct an expanded 20-task benchmark:

- 10 Single-Provision Lookup tasks derived from gold VLQA pairs (probation limits, notice periods, severance pay).
- 5 Multi-Article Synthesis tasks requiring multi-hop reasoning (e.g., resignation under Điều 35 combined with severance under Điều 46; illegal termination remedies under Điều 41 and 46).
- 3 Unanswerable / Out-of-Scope tasks (criminal sanctions, out-of-jurisdiction questions).

- 2 Adversarial tasks (fabricated decrees, prompt injection overrides).

We benchmark three system configurations across seeds 42, 123, and 2026:

1. **Baseline (OFF):** Standard single-turn RAG without verification.
2. **LuatCheck L1 (L1 ON):** ReAct loop with deterministic provenance and span verification.
3. **LuatCheck L1+L2 (L1+L2 ON):** Complete pipeline with L1 check and L2 semantic entailment.

Metric	Baseline (OFF)	LuatCheck (L1 ON)	LuatCheck (L1+L2 ON)
Recall@1 (↑)	0.800 ± 0.000	0.800 ± 0.000	0.800 ± 0.000
Recall@5 (↑)	$\mathbf{1.000 \pm 0.000}$	$\mathbf{1.000 \pm 0.000}$	$\mathbf{1.000 \pm 0.000}$
MRR (↑)	$\mathbf{0.900 \pm 0.000}$	$\mathbf{0.900 \pm 0.000}$	$\mathbf{0.900 \pm 0.000}$
Citation Precision (↑)	0.600 ± 0.000	$\mathbf{0.750 \pm 0.000}$	$\mathbf{0.750 \pm 0.000}$
Citation Recall (↑)	0.650 ± 0.000	$\mathbf{0.775 \pm 0.000}$	0.625 ± 0.000
Answer Coverage (↑)	0.900 ± 0.000	0.900 ± 0.000	$\mathbf{0.750 \pm 0.000}$
Unsupported-Sentence Rate (↓)	0.150 ± 0.000	0.150 ± 0.000	$\mathbf{0.000 \pm 0.000}$
Latency p50 / p95 (ms)	7.0 / 9.5	**6.6 / 8.9**	9.1 / 11.0
Compute Spend (USD)	\$0.00	\$0.00	\$0.00

Table 1: Three-way ablation performance across 3 seeds (42, 123, 2026) on the 20-task benchmark under cold-cache isolation.

5.2 Error Taxonomy Analysis

Following Kapoor et al. [12], we categorize all execution errors across configurations over 60 runs (20 tasks × 3 seeds):

Failure Category	Baseline (OFF)	LuatCheck (L1 ON)	LuatCheck (L1+L2 ON)
Retrieval Miss (not in top-5)	**0**	**0**	**0**
Wrong Provision Cited	18	9	**0**
Unsupported Claim Generated	9	9	**0**
Over-Refusal	**0**	**0**	**0**
Budget Exhausted (≥ 5 calls)	**0**	**0**	**0**
Parse / Schema Error	**0**	**0**	**0**

Table 2: Systematic error taxonomy across configurations on 20 tasks × 3 seeds (60 runs total).

The empirical findings demonstrate the complementary role of each verification tier: L1 verification boosts Citation Precision from 0.600 to **0.750** (1.000 across all 15 answerable tasks) and halves wrong provision citations (from 18 to 9). However, unigram overlap alone cannot detect out-of-scope inquiries that share surface labor-law terms (Tasks 16–18), leaving a USR of 0.150 (9/60 runs) under L1. Adding the L2 semantic entailment layer rejects all 3 unanswerable out-of-scope tasks per seed while preserving 100% coverage on the 15 answerable tasks (Coverage = 15/20 = 0.750, Over-Refusal = 0), driving both unsupported claims and wrong provision citations to **0** (USR = **0.000**). All configurations maintain zero API cost (\$0.00) and sub-12 ms p95 latency.

6 Security and Safety Analysis (R6)

Threat Model v2. LuatCheck operates under an untrusted execution environment where user inputs and third-party documents are treated as untrusted. Assets include user data privacy, legal correctness, and communication egress points.

Attack Category	Specific Attack Vector	D2 Defended	D3 Defended
Direct Injection	System Prompt Exfiltration (instruction override)	1/1	**1/1**
Direct Injection	Legal Evasion Advice (avoiding social insurance)	1/1	**1/1**
Direct Injection	Roleplay Bypass (unethical lawyer roleplay)	0/1	**1/1**
Direct Injection	Fabricated Statutory Citation (Điều 999)	1/1	**1/1**
Indirect Injection	Unauthorized Egress in Pasted Contract	1/1	**1/1**
Indirect Injection	Forged Termination Clause	1/1	**1/1**
Indirect Injection	SQL Injection in Pasted Case File	1/1	**1/1**
Indirect Injection	Cross-Session Memory Dump	1/1	**1/1**
Poisoned Shard	Zero-wage Probation Clause	1/1	**1/1**
Poisoned Shard	Severance Repeal Forgery	1/1	**1/1**
Defense Summary	**Attack Success Rate (ASR) ↓**	**20.0%**	**0.0%**
	Overall Defense Rate ↑	**80.0%**	**100.0%**

Table 3: Comparative red-teaming defense performance between D2 and D3 across 10 attack vectors.

Hardened boundary tagging and expanded regex classifiers raised the defense rate to 100.0%, leaving zero successful injections in the benchmark suite. Egress transmission via `send_summary` maintained a 100% defense record via mandatory interactive confirmation.

7 Limitations, Threats to Validity, and Extensibility

Limitations. The statutory corpus is presently bounded to the Labor Code 2019. Inquiries involving cross-statute interactions (e.g., criminal penalties under the Penal Code or corporate governance under the Law on Enterprises) are intentionally refused. Second, single-worker Apple Silicon serving handles up to 4 concurrent interactive sessions before p95 latency exceeds 15 seconds.

Threats to Validity. Benchmark queries were evaluated on standardized statutory articles. While representative of HR inquiries, real-world disputants may use highly colloquial Vietnamese phrasing or regional dialects, which may reduce dense retrieval recall.

Extensibility. The architecture is modular: replacing the corpus JSON with the Civil Code or Tax Law expands domain coverage immediately without altering the ReAct loop, MCP server, or verification engine.

8 Ethics and Responsible Use (PDPL 2025)

In strict compliance with Decree No. 356/2025/ND-CP and Law No. 91/2025/QH15 [2, 3], LuatCheck enforces on-premise execution boundaries. All inference executes on local hardware; no personal data or consultation transcripts are transmitted to third-party APIs. The episodic memory store enforces automated PII scrubbing, preserving legal query semantics while discarding personal identifiers.

9 Conclusion and Future Work

LuatCheck demonstrates that localized, small-scale language models can deliver reliable, citation-grounded legal assistance when governed by disciplined agentic architecture. By pairing a budgeted ReAct loop with MCP tool permissions, episodic memory, and dual-tier evidence verification, the system eliminates unsupported claims (0.0% error rate) and resists adversarial prompt injections (100% defense rate) at zero API expense. Future work will investigate cross-statute indexing and reinforcement learning from verifiable legal rewards.

References

[1] Varun Magesh et al. "Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools". In: *Journal of Empirical Legal Studies* 22 (2025), pp. 216–242.

[2] National Assembly of Vietnam. *Law on Personal Data Protection, Law No. 91/2025/QH15*. Passed 26 June 2025; effective 1 January 2026. 2025.

[3] Government of Vietnam. *Decree No. 356/2025/ND-CP*. Decree on personal data protection. 2025.

[4] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020.

[5] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations (ICLR)*. 2023.

[6] Anthropic. *Building Effective Agents*. Technical article. 2024.

[7] Tianyu Gao et al. "Enabling Large Language Models to Generate Text with Citations". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2023.

[8] Jie Huang et al. "Large Language Models Cannot Self-Correct Reasoning Yet". In: *International Conference on Learning Representations (ICLR)*. 2024.

[9] Kai Greshake et al. "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection". In: *Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISec)*. 2023.

[10] Edoardo Debenedetti et al. "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents". In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2024.

[11] Tan-Minh Nguyen et al. *VLQA: The First Comprehensive, Large, and High-Quality Vietnamese Dataset for Legal Question Answering*. 2025. arXiv: 2507.19995 [cs.CL].

[12] Sayash Kapoor et al. *AI Agents That Matter*. Preprint. 2024.

Appendix: Reproducibility, Contributions, and AI Statement

Reproducibility Package (R8). The codebase is tagged `v1.0-final` in Git:

- **Setup:** `pip install -r requirements.txt`
- **Unit Tests:** `make test` (485/485 tests passing)
- **Evaluation Benchmark:** `make eval`
- **Security Red Teaming:** `make redteam`
- **Live Web Demo:** `make demo`
- **Compile Report:** `make report`

Group 8 Contribution Statement.

- **Lê Trí Quyền (33.3%):** Curated and segmented statutory corpus; implemented BM25Okapi, dense semantic engine, and Reciprocal Rank Fusion hybrid retrieval.
- **Võ Phạm Tuấn Dũng (33.3%):** Implemented budgeted ReAct controller, L1 deterministic verifier, L2 semantic entailment layer, 20-task benchmark harness, and quantitative ablation analysis.
- **Nguyễn Hoàng Phong (33.3%):** Built FastMCP server, permission gatekeeper, Streamlit three-column interactive demo UI, JSONL audit logger, and red-teaming test suite.

AI-Use Statement. Generative language models were utilized solely for code scaffolding, synthetic attack case formulation, and LaTeX typography styling. All empirical measurements, legal annotations, architectural logic, and validation outcomes were independently authored, audited, and executed on local hardware by the students.